GARANTIA DA QUALIDADE DE SOFTWARE · TRABALHO INTEGRADOR

Financy

Sistema de Folha de Pagamento

Avaliação A1 · Planejamento, Modelagem e Garantia da Qualidade

Campo	Informação
Projeto	Financy · Sistema de Folha de Pagamento
Equipe	Ana Clara S. F. Gomes Iago Ferst Tubiana João Vitor Rosiak de O. Baltezan Mariana F. Hammel da Silva
Data	20/10/2026
Versão	1.0 · Entrega da A1
Repositório	github.com/johnrosik/folha-pagamento-gqs

Sumário

1. Documento de Visão

2. Especificação de Requisitos de Software (ERS)

3. Modelagem UML

4. Documento de Arquitetura

5. Plano de Garantia da Qualidade (PGQ)

6. Plano de Testes

7. Especificação de Casos de Teste

8. Planejamento do Projeto

9. Repositório do Projeto

Anexo · Ata de revisão técnica

1. Documento de Visão

1.1 Objetivo do sistema

Automatizar o processo de fechamento da folha de pagamento por meio de um sistema seguro que gerencia diferentes modelos de contratação (padrão, comissionado e produção). O software tem como objetivo eliminar o cálculo manual de remunerações e impostos, centralizando a emissão de demonstrativos individuais e relatórios gerenciais consolidados para otimizar a rotina administrativa.

1.2 Escopo do projeto

Dentro do escopo:

- cadastro de colaboradores dos três tipos: padrão (mensalista), comissionado e por produção;
- consulta, alteração e exclusão de cadastros;
- cálculo automático do salário bruto de cada tipo;
- cálculo de INSS, IRRF e salário líquido, como funcionalidade extra além do pedido pelo cliente (RF010), com alíquotas simplificadas (ver Premissas);
- folha de pagamento individual e resumo consolidado com o total geral da folha;
- gravação dos dados em banco SQLite;
- duas interfaces: menu de terminal e página web.

Fora do escopo:

- pagamento ou integração com bancos e com o eSocial;
- férias, 13º salário, FGTS, horas extras, benefícios e rescisão;
- histórico de folhas por mês;
- controle de acesso com login e vários perfis de usuário;
- tabelas oficiais progressivas de INSS e IRRF.

1.3 Partes interessadas (stakeholders)

Stakeholder	Papel	Interesse
Cliente	Empresa que contrata o sistema	Folha calculada sem erro de conta e custo total da folha visível num só relatório
Usuário	Gestor da folha de pagamento, que opera o sistema	Cadastrar e manter colaboradores com rapidez e emitir folha e resumo sem cálculo manual
Equipe	Desenvolvimento, qualidade e documentação	Entregar o sistema e os artefatos com evidência de qualidade, nos prazos da A1 e da A3
Professor	Avaliador do trabalho	Conformidade com o enunciado e evidências objetivas de qualidade

1.4 Premissas

- O cálculo de INSS e IRRF é simplificado, para fins acadêmicos: INSS de 10% sobre o salário bruto; IRRF de 7,5% sobre (bruto − INSS) quando essa base passa de R$ 2.259,20, e zero abaixo disso. Não são as tabelas oficiais de 2026.
- O sistema calcula a folha de um único período por vez, sem guardar histórico de meses anteriores.
- Há um único tipo de usuário, o gestor da folha, que usa o sistema na própria máquina.
- Os dados informados pelo gestor (vendas, quantidade produzida, salário) são verdadeiros; o sistema valida apenas formato e as regras RN001 a RN008.
- As máquinas da equipe e do avaliador têm Python 3.12 ou superior e um navegador atual.

1.5 Restrições

- Prazos: entrega e apresentação da A1 em 20/10/2026, às 19h; entrega da A3 em 01/12/2026.
- Equipe de quatro integrantes, com dedicação parcial.
- Tecnologias gratuitas e de código aberto: Python, Flask, SQLite.
- Execução local, sem servidor na internet.
- Entrega organizada na estrutura de pastas 01 a 11 definida no enunciado.

1.6 Benefícios esperados

- Fim do cálculo manual da folha e dos erros de conta que vêm dele.
- Dados inválidos (valores negativos, nome vazio, matrícula repetida) são barrados antes de qualquer cálculo.
- Visão consolidada do custo da folha, com o total geral em um só relatório.
- Registros preservados entre um uso e outro do sistema.
- Novos tipos de remuneração podem ser incluídos sem alterar os cálculos existentes (RNF004).
- Rastreabilidade entre requisito, caso de teste e código, o que facilita a manutenção e a homologação.

2. Especificação de Requisitos de Software (ERS)

Requisitos do cliente para o Sistema de Folha de Pagamento, cada um com identificador próprio, usado na matriz de rastreabilidade e nos casos de teste.

2.1 Requisitos funcionais

ID	Descrição
RF001	Cadastro de Colaborador
RF002	Cadastro de Colaborador Padrão
RF003	Cadastro de Colaborador Comissionado
RF004	Cadastro de Colaborador por Produção
RF005	Consulta de Colaboradores
RF006	Cálculo de Salário
RF007	Geração da Folha de Pagamento
RF008	Emissão de Resumo da Folha
RF009	Manutenção de Cadastros
RF010	Cálculo de descontos (INSS e IRRF) e do salário líquido (funcionalidade extra)

2.2 Regras de negócio

ID	Descrição
RN001	Toda matrícula deverá ser única dentro do sistema.
RN002	O nome do colaborador é obrigatório.
RN003	O salário base não poderá ser negativo.
RN004	O valor de vendas não poderá ser negativo.
RN005	O percentual de comissão não poderá ser negativo.
RN006	A quantidade produzida não poderá ser negativa.
RN007	O valor pago por unidade produzida não poderá ser negativo.
RN008	O salário final deverá ser calculado automaticamente pelo sistema.

2.3 Requisitos não funcionais

ID	Descrição
RNF001	O sistema deverá possuir interface amigável e intuitiva.
RNF002	O sistema deverá validar os dados de entrada antes de efetuar qualquer cálculo.
RNF003	O sistema deverá armazenar os registros de colaboradores de forma persistente.
RNF004	O sistema deverá permitir futura inclusão de novos tipos de remuneração sem necessidade de alterar os cálculos já existentes.

2.4 Critérios de aceitação

Requisito	Critério de aceitação
RF001	O sistema permite cadastrar os três tipos de colaborador (padrão, comissionado e por produção), com matrícula única e nome obrigatório.
RF002	O sistema cadastra o colaborador padrão com salário base maior ou igual a zero; o salário bruto é o próprio salário base.
RF003	O sistema cadastra o comissionado com salário base, vendas e percentual de comissão, nenhum negativo; bruto = salário base + vendas × comissão ÷ 100.
RF004	O sistema cadastra o colaborador por produção com quantidade produzida e valor por unidade, nenhum negativo; bruto = quantidade × valor por unidade.
RF005	O sistema consulta o colaborador pela matrícula e informa quando nenhum colaborador é encontrado.
RF006	O sistema calcula o salário bruto conforme o tipo de remuneração, sem cálculo manual (RN008).
RF007	O sistema gera a folha individual de um colaborador, com salário bruto, descontos e salário líquido.
RF008	O sistema apresenta o resumo consolidado com todos os colaboradores e o total geral da folha.
RF009	O sistema permite alterar e excluir cadastros, e a mudança continua gravada depois de fechar e abrir o sistema.
RF010	INSS = 10% do bruto; IRRF = 7,5% sobre (bruto − INSS) quando essa base passa de R$ 2.259,20; líquido = bruto − INSS − IRRF.

2.5 Matriz de rastreabilidade dos requisitos

Status possíveis: Não iniciado, Em andamento, Aprovado, Reprovado. Na A1 nenhum teste foi executado; o status muda na A3, quando cada caso de teste for executado.

Requisito	Descrição	Regras associadas	Casos de teste	Status
RF001	Cadastro de colaborador	RN001, RN002	CT001, CT010, CT011	Não iniciado
RF002	Colaborador padrão	RN003, RN008	CT002, CT012	Não iniciado
RF003	Colaborador comissionado	RN003, RN004, RN005, RN008	CT003, CT013, CT014, CT017	Não iniciado
RF004	Colaborador por produção	RN006, RN007, RN008	CT004, CT015, CT016, CT018	Não iniciado
RF005	Consulta de colaboradores	RN001	CT005	Não iniciado
RF006	Cálculo de salário	RN003 a RN008	CT006, CT017, CT018	Não iniciado
RF007	Folha de pagamento individual	RN008	CT007	Não iniciado
RF008	Resumo da folha	RN008	CT008, CT019	Não iniciado

Requisito	Descrição	Regras associadas	Casos de teste	Status
RF009	Manutenção de cadastros	RN001, RN002	CT009	Não iniciado
RF010	Descontos e salário líquido	RN008	CT017, CT018, CT019	Não iniciado
RNF003	Persistência	—	CT020	Não iniciado

3. Modelagem UML

Diagramas escritos em PlantUML; o código-fonte de cada um está na pasta 02_Analise do repositório.

3.1 Diagrama de casos de uso

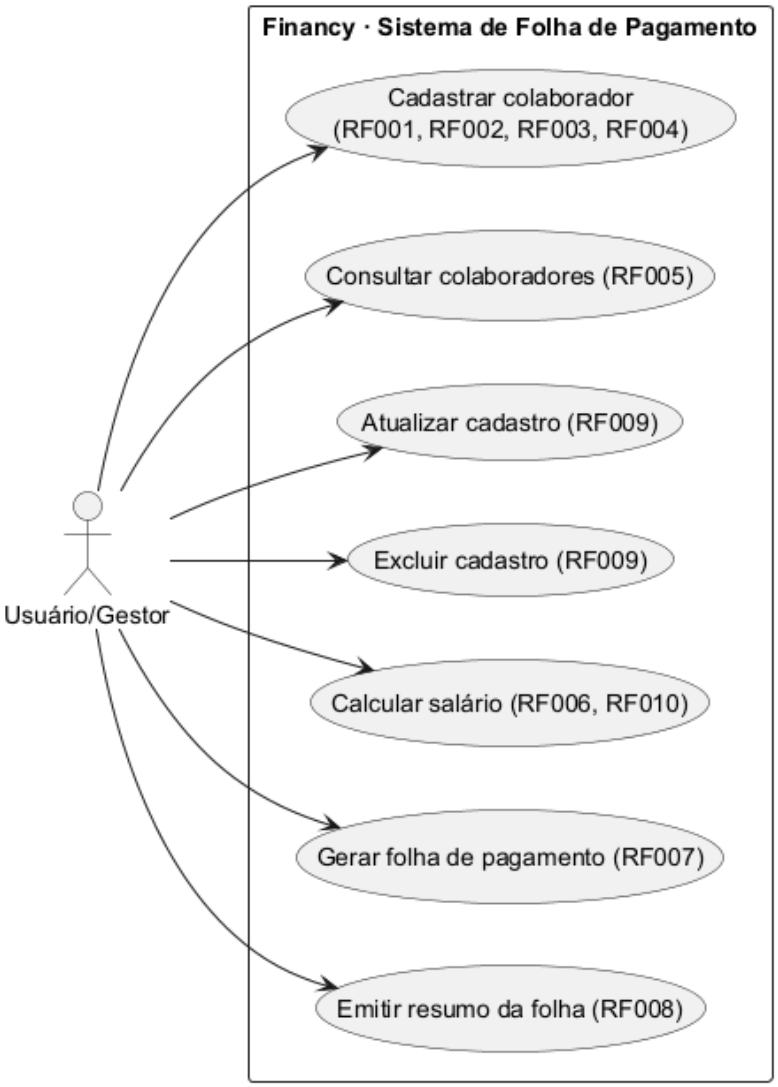


Figura 1 · Casos de uso do Usuário/Gestor

Código	Caso de uso	Ator	Objetivo
UC01	Cadastrar colaborador	Usuário/Gestor	Registrar colaborador padrão, comissionado ou por produção (RF001 a RF004)
UC02	Consultar colaboradores	Usuário/Gestor	Localizar um colaborador pela matrícula (RF005)
UC03	Atualizar cadastro	Usuário/Gestor	Alterar os dados de um colaborador (RF009)
UC04	Excluir cadastro	Usuário/Gestor	Remover um colaborador (RF009)

Código	Caso de uso	Ator	Objetivo
UC05	Calcular salário	Usuário/Gestor	Obter o salário bruto, os descontos e o líquido (RF006, RF010)
UC06	Gerar folha de pagamento	Usuário/Gestor	Emitir a folha individual de um colaborador (RF007)
UC07	Emitir resumo da folha	Usuário/Gestor	Emitir o resumo consolidado com o total geral (RF008)

3.2 Diagrama de classes

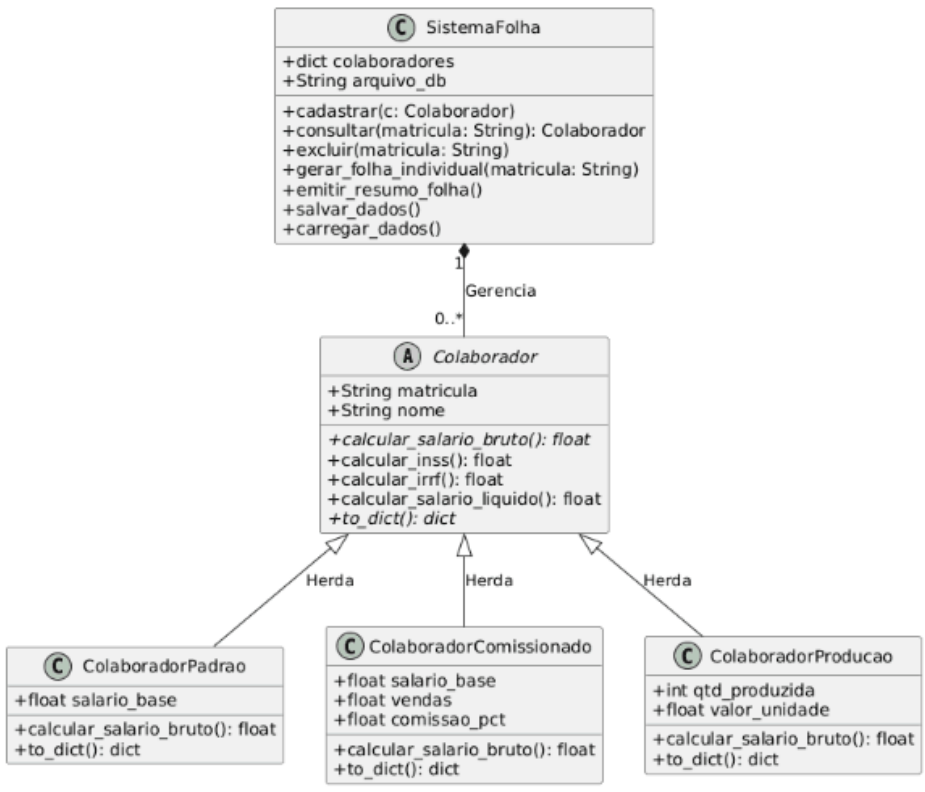


Figura 2 · Colaborador é abstrata; cada subclasse implementa o próprio cálculo do salário bruto (RNF004)

3.3 Diagrama de sequência

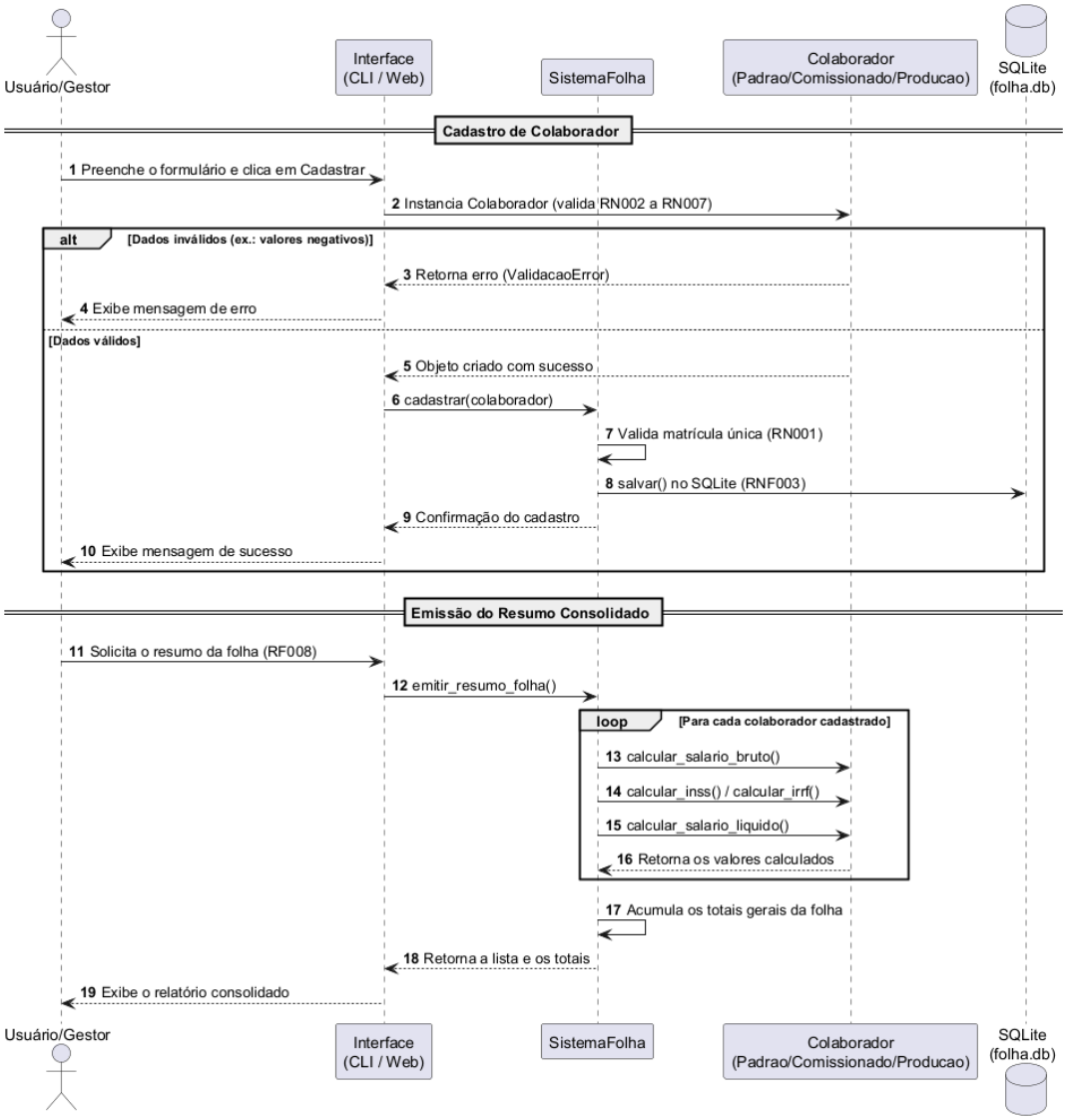


Figura 3 · Cadastro com validação e emissão do resumo consolidado

3.4 Diagrama de atividades

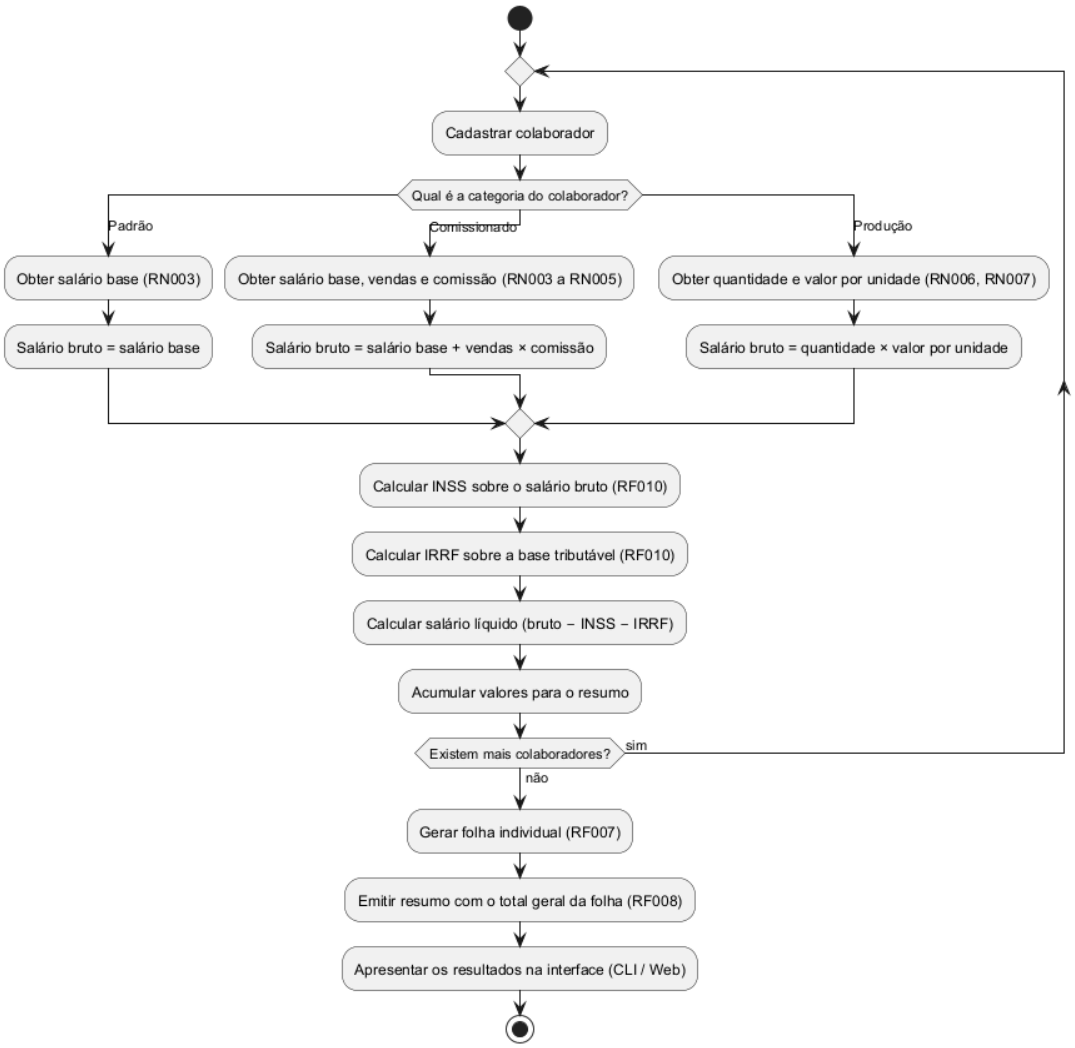


Figura 4 · Fluxo do cálculo da folha, do cadastro ao resumo

4. Documento de Arquitetura

4.1 Arquitetura adotada

O sistema adota uma arquitetura em camadas, em que cada camada só conhece a camada imediatamente abaixo dela:

Camada	Responsabilidade	Componentes
Apresentação	Receber os dados do usuário e exibir resultados	Menu de terminal (CLI) e página web (index.html)
Aplicação	Expor e orquestrar os casos de uso: cadastrar, consultar, alterar, excluir, gerar folha individual e resumo consolidado	SistemaFolha e API REST em Flask
Domínio	Regras de negócio (RN001 a RN008) e cálculo da remuneração	Colaborador (abstrata), ColaboradorPadrao, ColaboradorComissionado, ColaboradorProducao, ValidacaoError
Persistência	Gravar e restaurar os colaboradores	Repositório SQLite

As duas interfaces usam o mesmo SistemaFolha, então uma regra de negócio é implementada uma única vez e vale para o terminal e para a web. A camada de domínio não conhece banco de dados nem interface, o que permite testá-la isoladamente.

A extensibilidade exigida pelo RNF004 vem do princípio aberto/fechado: um novo tipo de remuneração é uma nova subclasse de Colaborador que implementa calcular_salario_bruto(), sem alterar os cálculos existentes.

Situação do código herdado do T1: o domínio e o menu de terminal existem; a página web já chama a API em http://127.0.0.1:5000/api/colaboradores, mas o servidor Flask ainda não existe; a persistência atual é em arquivo JSON e passa para SQLite na A3.

4.2 Tecnologias selecionadas

Camada	Tecnologia
Front-end	HTML5, CSS e JavaScript (Fetch API), sem framework
Back-end	Python 3.12 ou superior, Flask 3
Banco	SQLite 3, pelo módulo sqlite3 da biblioteca padrão do Python
Testes	pytest, pytest-cov e o cliente de testes do Flask
Qualidade de código	ruff (verificação de estilo e formatação)
Versionamento e gestão	Git, GitHub (Issues e Projects)
Modelagem	PlantUML

4.3 Justificativas técnicas

- **Python:** a equipe já domina a linguagem, o T1 foi escrito nela e a orientação a objetos com classes abstratas (abc) representa bem a hierarquia de colaboradores.

- **Flask:** a API precisa de poucas rotas. Um microframework atende com pouco código e tem cliente de testes próprio, usado nos testes de integração.
- **SQLite:** já vem com o Python, não exige servidor nem instalação na máquina de quem corrige, guarda tudo num arquivo só e aceita SQL padrão, o que permite versionar o script de criação do banco na pasta 06_Banco_Dados.
- **HTML e JavaScript sem framework:** a página roda direto no navegador, sem etapa de compilação.
- **pytest e pytest-cov:** padrão de mercado em Python, com fixtures para isolar cada teste e relatório de cobertura para medir a meta de 80%.
- **GitHub Issues e Projects:** concentram tarefas, defeitos e cronograma no mesmo lugar do código, o que dá rastreabilidade entre requisito, issue, commit e Pull Request.

4.4 Organização dos componentes

Estrutura prevista para 05_Codigo_Fonte na A3:

Arquivo	Conteúdo	Depende de
dominio.py	Colaborador, subclasses, ValidacaoError, cálculos	nada
repositorio.py	Gravação, leitura e exclusão no SQLite	dominio.py
sistema.py	SistemaFolha: casos de uso, folha individual e resumo consolidado	dominio.py, repositorio.py
cli.py	Menu de terminal	sistema.py
api.py	Rotas Flask	sistema.py
web/index.html	Página web	api.py (via HTTP)

Rotas da API:

Método e rota	Caso de uso
GET /api/colaboradores	Consultar colaboradores e resumo (RF005, RF008)
GET /api/colaboradores/<matricula>	Folha individual (RF006, RF007)
POST /api/colaboradores	Cadastrar (RF001 a RF004)
PUT /api/colaboradores/<matricula>	Alterar (RF009)
DELETE /api/colaboradores/<matricula>	Excluir (RF009)

Erros de validação (ValidacaoError) viram resposta HTTP 400 com a mensagem da regra violada; matrícula inexistente vira 404.

Os relatórios passam a ser montados em SistemaFolha como dados (listas e totais) e formatados por cada interface. Hoje gerar_folha_individual e emitir_resumo_folha imprimem direto no terminal, o que impede a web de reaproveitá-los.

4.5 Estratégia de persistência

Os colaboradores ficam numa única tabela SQLite, com uma coluna tipo que indica a categoria (herança em tabela única). As regras de negócio também são garantidas pelo banco, como segunda barreira:

Coluna	Tipo	Restrição	Regra
matricula	TEXT	chave primária	RN001
nome	TEXT	não nulo e não vazio	RN002
tipo	TEXT	Padrao, Comissionado ou Producao	RF002 a RF004
salario_base	REAL	maior ou igual a zero	RN003
vendas	REAL	maior ou igual a zero	RN004
comissao_pct	REAL	maior ou igual a zero	RN005
qtd_produzida	INTEGER	maior ou igual a zero	RN006
valor_unidade	REAL	maior ou igual a zero	RN007

- O salário bruto, o INSS, o IRRF e o líquido não são gravados: são calculados a cada consulta (RN008), então nunca ficam desatualizados.
- O script 06_Banco_Dados/schema.sql cria a tabela, e o sistema o executa na inicialização se a tabela ainda não existir.
- Toda consulta usa parâmetros (?) em vez de montar o SQL com texto, o que impede injeção de SQL.
- Cada operação de escrita é confirmada numa transação; se falhar, nada é gravado pela metade.
- Na carga, um registro inválido é informado com a matrícula e o motivo, e os demais continuam sendo carregados. No T1, um registro inválido interrompe a carga de todos os seguintes, com um aviso genérico que não diz qual registro falhou (folhadepagamento.py, linhas 242 e 243).
- Os testes usam um banco temporário próprio, nunca o arquivo de uso real.

5. Plano de Garantia da Qualidade (PGQ)

5.1 Padrões de codificação

- Estilo PEP 8, verificado pelo ruff, com linhas de até 100 caracteres. Código com aviso do ruff não é mesclado.
- Nomes em português: classes em PascalCase (ColaboradorPadrao); funções, métodos e variáveis em snake_case (calcular_salario_bruto); constantes em maiúsculas.
- Anotação de tipo em parâmetros e retornos.
- Docstring em toda classe e método público, citando o requisito ou a regra que implementa (exemplo: "RN003: salário base não pode ser negativo.").
- Violação de regra de negócio lança ValidacaoError com o código da regra na mensagem (exemplo: "RN001: …").
- A camada de domínio não imprime nada nem acessa banco; impressão e leitura de tela ficam só nas interfaces.
- Valores monetários são exibidos com duas casas decimais.
- Nenhuma exceção é silenciada: todo except informa o que falhou.

5.2 Controle de versões

- Repositório Git no GitHub: johnrosik/folha-pagamento-gqs, com a estrutura de entrega de 01 a 11.
- O ramo main guarda sempre uma versão que executa. Ninguém trabalha direto nele.
- Cada tarefa é feita num ramo próprio, nomeado pelo tipo e pela issue: feat/12-cadastro-sqlite, fix/30-irrf-negativo, docs/9-readme.
- Mensagens de commit em português, no imperativo, com o número da issue: "Valida matrícula duplicada no banco (#12)".
- Toda mudança entra por Pull Request, revisado e aprovado por outro integrante antes da mesclagem. A mesclagem preserva os commits, para o histórico mostrar o trabalho de cada um.
- Tarefas, defeitos e cronograma ficam nas Issues e no GitHub Projects; todo Pull Request cita a issue que resolve.

5.3 Processo de revisão técnica

Inspeção de código. Certificar que o código executa todos os requisitos e regras; verificar possíveis erros, bugs e outros problemas; garantir que o front-end está conectado ao back-end, sem função sem representação visual.

Artefatos e software. Analisar a qualidade da documentação técnica e dos modelos conceituais; revisar a correspondência entre o código-fonte e os diagramas UML; o software precisa estar de acordo com a definição técnica do projeto, seguindo as regras e a organização definidas pela equipe.

5.4 Critérios de qualidade

Os critérios de qualidade deste projeto seguem a norma ISO/IEC 25010.

Característica	Critério no projeto
Adequação funcional	Executar todos os requisitos funcionais e os cálculos de salário, desconto e valores sem erro matemático.
Eficiência de desempenho	Cálculo, registro e resumo executados com rapidez e baixo consumo de memória e processamento.
Compatibilidade	Rodar em conjunto com outros programas e nos sistemas operacionais e ambientes da equipe.
Usabilidade	Menus e formulários com linguagem simples; interface limpa; entradas validadas antes dos cálculos.
Confiabilidade	Guardar e restaurar dados em caso de erro, falha técnica ou desligamento; exceções tratadas sem encerrar a aplicação.
Segurança	Dados mantidos com integridade dos registros.
Manutenibilidade	Permitir atualizações, modificações e adaptações.
Portabilidade	Ser transferido de um ambiente de hardware, software ou sistema operacional para outro.

5.5 Métricas adotadas

Métrica	Como se mede	Meta	Ferramenta
Cobertura de testes	linhas executadas pelos testes ÷ linhas do código	≥ 80%	pytest-cov
Requisitos atendidos	RF com caso de teste aprovado ÷ total de RF	100%	matriz de rastreabilidade
Casos de teste aprovados	aprovados ÷ executados	100% na homologação	relatório de testes
Defeitos críticos abertos	issues bug com severidade crítica e abertas	0	GitHub Issues
Taxa de correção	defeitos corrigidos ÷ defeitos encontrados	100% dos críticos e altos	GitHub Issues
Densidade de defeitos	defeitos encontrados ÷ mil linhas de código	acompanhamento	GitHub Issues
Conformidade de estilo	avisos do ruff	0	ruff

As métricas são medidas a cada Pull Request (cobertura e estilo) e consolidadas no Relatório Final de Qualidade.

5.6 Critérios de aprovação

ID	Item de avaliação	Critério para aprovação	Peso
CA001	Requisitos	100% dos requisitos funcionais implementados e operacionais.	Obrigatório

ID	Item de avaliação	Critério para aprovação	Peso
CA002	Regras de negócio	Aprovação em todos os cenários de teste.	Obrigatório
CA003	Acurácia	O relatório exibe com precisão os cálculos dos salários.	Obrigatório
CA004	Persistência dos dados	Salvamento e restauração dos dados com sucesso.	Obrigatório
CA005	Qualidade da arquitetura	Aplicação total dos diagramas UML.	Importante

5.7 Fluxo da garantia da qualidade

1. Requisitos	2. Análise	3. Desenvolvimento	4. Testes	5. Homologação	6. Entrega
Visão e ERS revisadas	UML e arquitetura revisadas	Código por PR revisado	Unitários, integração e aceitação	Termo de homologação	Pastas 01 a 11

Cada etapa só começa quando a anterior passou pela revisão técnica descrita em 5.3.

6. Plano de Testes

6.1 Escopo

O foco principal da validação e dos testes são as funções de cadastro, o gerenciamento dos colaboradores e as formas de remuneração do projeto (colaboradores padrão, comissionados e por produção), de acordo com os requisitos funcionais.

A verificação também avalia todas as regras de negócio estabelecidas, com foco na matrícula do colaborador (RN001), na exatidão dos cálculos do salário bruto, dos descontos de INSS e IRRF e do salário líquido (RN008, RF010) e na emissão do resumo consolidado da folha (RF007 e RF008), garantindo a precisão dos somatórios e do total geral a ser pago pela empresa.

Ficam fora do escopo desta fase: segurança avançada, integração com bancos de dados complexos, testes de entrada extrema de dados e acessos simultâneos. A validação concentra-se no comportamento funcional e na usabilidade da aplicação.

6.2 Estratégia

A estratégia combina técnicas de engenharia de software para cobrir as regras de negócio e a arquitetura do sistema, em níveis crescentes:

- **Nível unitário:** testa os métodos de cálculo, verificando a exatidão das fórmulas do salário, das deduções e do valor líquido de cada tipo de colaborador.
- **Nível de integração:** confirma o fluxo de dados, em que as informações inseridas são convertidas e gravadas sem perda de integridade.
- **Sistema e aceitação:** avalia o software do ponto de vista do usuário final, percorrendo todos os fluxos e interfaces para confirmar que os casos de uso atendem aos requisitos.

6.3 Níveis de teste

Nível	O que se testa	Como
Unitário	Cálculos de cada tipo de colaborador, INSS, IRRF, líquido e cada regra RN001 a RN008, isoladamente	pytest, sem banco e sem interface
Integração	SistemaFolha com o SQLite (gravar, restaurar, excluir), as rotas da API e os relatórios	pytest com banco temporário e cliente de testes do Flask
Sistema	Fluxos completos pelo terminal e pela web: cadastrar, consultar, alterar, excluir, folha individual e resumo	Execução manual roteirizada pelos casos de teste
Aceitação	Os 20 casos de teste, do ponto de vista do cliente, com resultado esperado e obtido	Execução pela equipe no papel de cliente, registrada no Relatório de Testes

6.4 Ambiente

Item	Descrição
Sistema operacional	Windows 10 ou 11 (máquinas da equipe)
Linguagem	Python 3.12 ou superior
Banco	SQLite 3, arquivo local; nos testes, banco temporário criado e apagado por teste
Framework	Flask 3
Navegador	Google Chrome ou Microsoft Edge, versão atual
Dados de teste	Colaboradores fictícios criados pelos próprios testes

6.5 Ferramentas

Ferramenta	Uso
pytest	Execução dos testes unitários e de integração
pytest-cov	Medição da cobertura
Cliente de testes do Flask	Testes das rotas da API sem subir servidor
ruff	Verificação de estilo
GitHub Issues	Registro e acompanhamento de defeitos (etiqueta bug, severidade e prioridade)
GitHub Projects	Acompanhamento das tarefas e do cronograma
PlantUML	Diagramas UML

6.6 Critérios de entrada e saída

Critérios de entrada. Implementação do código-fonte concluída, com o back-end em Python e a modelagem orientada a objetos (classe Colaborador e subclasses Padrão, Comissionado e Produção); código inspecionado e revisado, sem erros que impeçam a execução; ambiente de testes configurado; cenários de validação definidos.

Critérios de saída. Testes planejados executados para todas as categorias de colaborador e para os relatórios individual e consolidado; nenhum defeito crítico, como erro matemático no cálculo do salário, falha na gravação ou interrupção da execução por dado incorreto; todas as regras de negócio testadas e atendidas.

6.7 Critérios de aprovação

O projeto será aprovado quando cumprir todos os requisitos de qualidade e todos os requisitos funcionais, com exatidão nos cálculos, persistência e restauração dos dados funcionando e tratamento comprovado das exceções de negócio, sem comprometer a estabilidade da aplicação. O software também precisa manter a integridade dos dados, seguir os padrões de arquitetura e estar de acordo com os documentos do sistema: requisitos, regras, diagramas e objetivos do projeto.

7. Especificação de Casos de Teste

Vinte casos de teste, cada um ligado a um requisito ou regra da matriz de rastreabilidade (2.5). Os valores esperados de CT017 a CT019 foram calculados com as regras das premissas (1.4).

ID	Requisito	Objetivo	Passos	Resultado esperado
CT001	RF001	Verificar se o sistema realiza corretamente o cadastro de um colaborador.	Informar matrícula, nome e tipo de colaborador. Consultar no sistema.	O sistema cadastra e armazena o colaborador corretamente.
CT002	RF002	Validar o cadastro do colaborador padrão com salário positivo.	Selecionar a categoria padrão e informar salário positivo.	Cadastro aceito com o salário base válido.
CT003	RF003	Testar a aceitação dos dados de um colaborador comissionado.	Selecionar comissionado, inserir salário, vendas e percentual. Salvar.	Cadastro aceito com salário base, vendas e percentual de comissão.
CT004	RF004	Cadastrar colaborador por produção a partir da quantidade e do valor por unidade.	Informar quantidade produzida e valor por unidade. Executar o cadastro.	Cadastro aceito e salário calculado como quantidade × valor por unidade.
CT005	RF005	Testar a busca de um colaborador pela matrícula.	Acessar a consulta, digitar a matrícula e ver o resultado.	O colaborador é localizado e os detalhes são exibidos.
CT006	RF006	Confirmar a exatidão dos cálculos do sistema.	Cadastrar colaboradores e verificar os valores calculados.	Cálculos feitos de forma automática e precisa.
CT007	RF007	Gerar e exibir a folha de pagamento individual.	Acessar a geração de folha, informar a matrícula e solicitar a emissão.	Relatório individual com nome, salário bruto, INSS, IRRF e líquido.
CT008	RF008	Gerar o relatório consolidado e conferir o total geral.	Emitir o resumo consolidado e visualizar a listagem de colaboradores.	Todos os registros listados, com o somatório dos salários e dos impostos.
CT009	RF009	Testar a exclusão de um registro e a atualização da persistência.	Excluir colaborador informando a matrícula e confirmar.	O cadastro é excluído e deixa de aparecer na consulta.
CT010	RN001	Verificar se o sistema impede duas matrículas iguais.	Cadastrar o padrão P01 (Ana Souza, R$ 3.000,00). Tentar cadastrar outro colaborador com a matrícula P01.	Segundo cadastro recusado com "RN001: Toda matrícula deverá ser única"; só o primeiro permanece.
CT011	RN002	Verificar se o sistema recusa nome vazio ou só com espaços.	Cadastrar padrão com matrícula P02, nome em branco e salário R$ 1.500,00.	Cadastro recusado com "RN002: O nome do colaborador é obrigatório."; nada é gravado.
CT012	RN003	Verificar se o sistema impede salário base negativo e exibe o erro.	Preencher matrícula e nome, selecionar o tipo e inserir salário negativo.	Cadastro barrado, com mensagem de erro na interface; o registro não é salvo.
CT013	RN004	Verificar se o sistema recusa valor de vendas negativo.	Cadastrar comissionado C02 com salário R$ 2.000,00,	Cadastro recusado com "RN004: O valor de

ID	Requisito	Objetivo	Passos	Resultado esperado
			vendas R$ −100,00 e comissão 5%.	vendas não pode ser negativo."
CT014	RN005	Verificar se o sistema recusa comissão negativa.	Cadastrar comissionado C03 com salário R$ 2.000,00, vendas R$ 10.000,00 e comissão −5%.	Cadastro recusado com "RN005: O percentual de comissão não pode ser negativo."
CT015	RN006	Verificar se o sistema recusa quantidade produzida negativa.	Cadastrar produção R02 com quantidade −10 e valor por unidade R$ 10,00.	Cadastro recusado com "RN006: A quantidade produzida não pode ser negativa."
CT016	RN007	Verificar se o sistema recusa valor por unidade negativo.	Cadastrar produção R03 com quantidade 100 e valor por unidade R$ −2,00.	Cadastro recusado com "RN007: O valor pago por unidade produzida não pode ser negativo."
CT017	RF006, RF010	Conferir bruto, INSS, IRRF e líquido de um comissionado, no limite de isenção do IRRF.	Cadastrar C01 (Bruno Lima): salário R$ 2.000,00, vendas R$ 10.000,00, comissão 5%. Gerar a folha individual.	Bruto R$ 2.500,00; INSS R$ 250,00; IRRF R$ 0,00 (base R$ 2.250,00); líquido R$ 2.250,00.
CT018	RF006, RF010	Conferir bruto, INSS, IRRF e líquido de um colaborador por produção, com IRRF.	Cadastrar R01 (Carla Dias): 500 unidades a R$ 10,00. Gerar a folha individual.	Bruto R$ 5.000,00; INSS R$ 500,00; IRRF R$ 337,50; líquido R$ 4.162,50.
CT019	RF008, RF010	Conferir os totais do resumo com os três tipos de colaborador.	Cadastrar P01 (padrão, R$ 3.000,00), C01 e R01. Emitir o resumo consolidado.	3 colaboradores; bruto R$ 10.500,00; INSS R$ 1.050,00; IRRF R$ 540,00; líquido R$ 8.910,00.
CT020	RNF003	Verificar se os dados continuam gravados após fechar e abrir o sistema.	Cadastrar P01, C01 e R01. Sair, abrir de novo e consultar as três matrículas.	As três matrículas são encontradas, com os mesmos dados do cadastro.

8. Planejamento do Projeto

8.1 Estrutura analítica do projeto (EAP)

Código	Pacote de trabalho	Entregáveis	Responsável
1	Projeto Sistema de Folha de Pagamento	Sistema e artefatos da A1 e da A3	Equipe
1.1	Gestão do projeto	EAP, cronograma, distribuição de responsabilidades, atas de revisão	Iago e Ana
1.2	Requisitos	Documento de Visão, ERS, matriz de rastreabilidade	Ana e Mariana
1.3	Análise e projeto	Diagramas UML, Documento de Arquitetura	Mariana e João
1.4	Garantia da qualidade	PGQ, Plano de Testes, casos de teste, registro de defeitos, métricas	Iago, João, Mariana e Ana
1.5	Construção	Código-fonte, banco de dados, API, interfaces de terminal e web	João
1.6	Testes	Testes unitários, de integração e de aceitação, relatório de testes	João e Ana
1.7	Documentação	README, Manual do Usuário, Manual Técnico	João e Mariana
1.8	Homologação e entrega	Termo de Homologação, Relatório Final de Qualidade, apresentações	Ana, Mariana e Iago

8.2 Cronograma

Etapa	Início	Fim	Responsável
Visão, ERS e planejamento	07/10	13/10	Ana, Mariana e Iago
UML, Arquitetura, PGQ e Plano de Testes	07/10	15/10	Mariana, João e Iago
Casos de teste	13/10	16/10	Mariana e Ana
Revisão cruzada da A1 (ata)	19/10	19/10	Iago
Entrega e apresentação da A1	20/10	20/10	Equipe
Implementação (RF001 a RF010, SQLite, API)	21/10	01/11	João
Testes unitários e de integração	02/11	08/11	João
Testes de aceitação e registro de defeitos	09/11	22/11	Ana e Iago
Manuais, métricas e relatórios	09/11	28/11	Mariana, João e Iago
Termo de homologação e apresentação	23/11	01/12	Ana e Iago
Entrega da A3	01/12	01/12	Equipe

O cronograma detalhado, item a item, está no GitHub Projects do repositório: github.com/users/johnrosik/projects/3.

8.3 Distribuição de responsabilidades

Integrante	Papel	A1	A3
João Vitor Rosiak de O. Baltezan	Desenvolvimento	Documento de Arquitetura, partes técnicas do PGQ e do Plano de Testes, repositório e README	Sistema, banco, testes unitários e de integração, Manual Técnico
Mariana F. Hammel da Silva	Requisitos e modelagem	ERS, diagramas UML, casos de teste CT001 a CT010	Relatório de Revisão Técnica, Manual do Usuário, Relatório Final de Qualidade
Iago Ferst Tubiana	Gestão e qualidade	PGQ, Plano de Testes, planejamento, ata da revisão cruzada	Registro de defeitos, métricas, relatório de testes, atas, apresentação
Ana Clara S. F. Gomes	Requisitos, testes e apresentação	Documento de Visão, casos de teste CT011 a CT020, montagem da entrega	Testes de aceitação, Termo de Homologação, apresentação, montagem da entrega

Cada artefato tem uma issue no repositório, com dono, apoio, prazo e checklist.

9. Repositório do Projeto

9.1 README

O README do repositório apresenta a equipe e os papéis, onde fica cada parte do trabalho, a estrutura de pastas, como executar o sistema, como contribuir e as datas de entrega. Endereço: github.com/johnrosik/folha-pagamento-gqs.

9.2 Histórico de commits

Toda mudança entra por Pull Request ligado a uma issue, e a mesclagem preserva os commits de cada integrante (5.2). O histórico completo está no próprio repositório.

9.3 Estrutura inicial do projeto

Pasta	Conteúdo
01_Requisitos	Documento de Visão, ERS, matriz de rastreabilidade
02_Analise	Diagramas UML (PlantUML e imagens)
03_Projeto	Documento de Arquitetura
04_Testes	Plano de Testes, casos de teste, relatórios de execução e de cobertura
05_Codigo_Fonte	Código do sistema (hoje, o código do T1)
06_Banco_Dados	Script de criação do banco SQLite
07_Executavel	Instruções e script para executar o sistema
08_Manuais	Manual do Usuário e Manual Técnico
09_Homologacao	Testes de aceitação e Termo de Homologação
10_Qualidade	PGQ, registro de defeitos, métricas, atas e Relatório Final de Qualidade
11_Apresentacao	Planejamento do projeto e slides

Anexo · Ata de revisão técnica

Modelo do Anexo I (seção 15), preenchido na revisão cruzada da A1.

Item	Informação
Projeto	Sistema de Folha de Pagamento
Data	
Participantes	
Artefatos revisados	
Não conformidades encontradas	
Correções combinadas	
Responsável e prazo	